

F8: Sekvensdiagram

DA343A, Institutionen för datateknik och medievetenskap

Innehåll

Repetition

Ny notation

Ex-tentor

Visual Paradigm

Sammanfattning



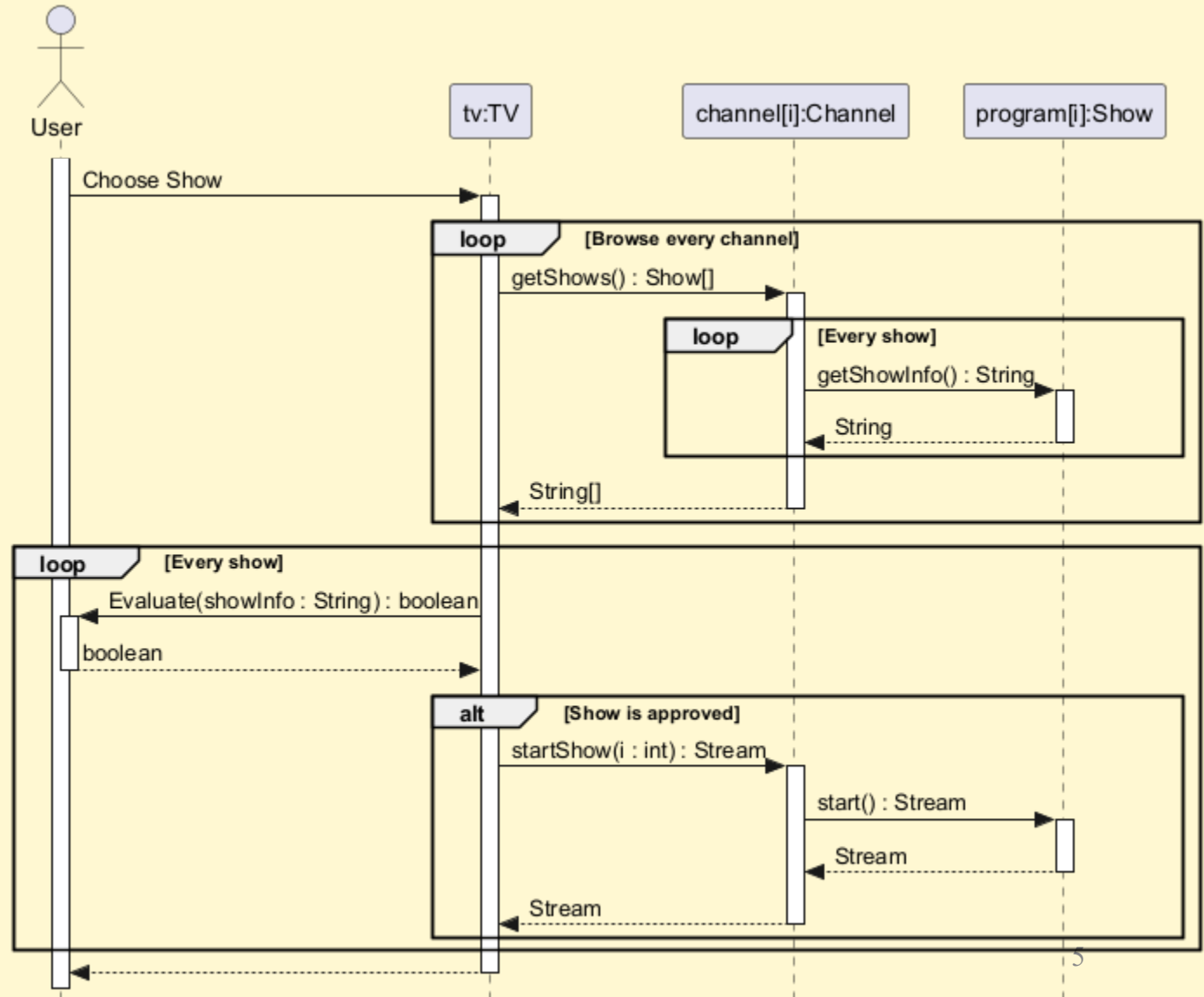
Repetition

Repetition

- Sekvensdiagram visar ett flöde av aktiviteter
- De visar i vilken ordning saker sker
- De visar kommunikation mellan objekt
- De visar vem/vad som utför en aktivitet
- De visar vem/vad som ”beställer” en aktivitet
- De visar vad för svar som förväntas mottagas
- De visar *inte* hur något utförs

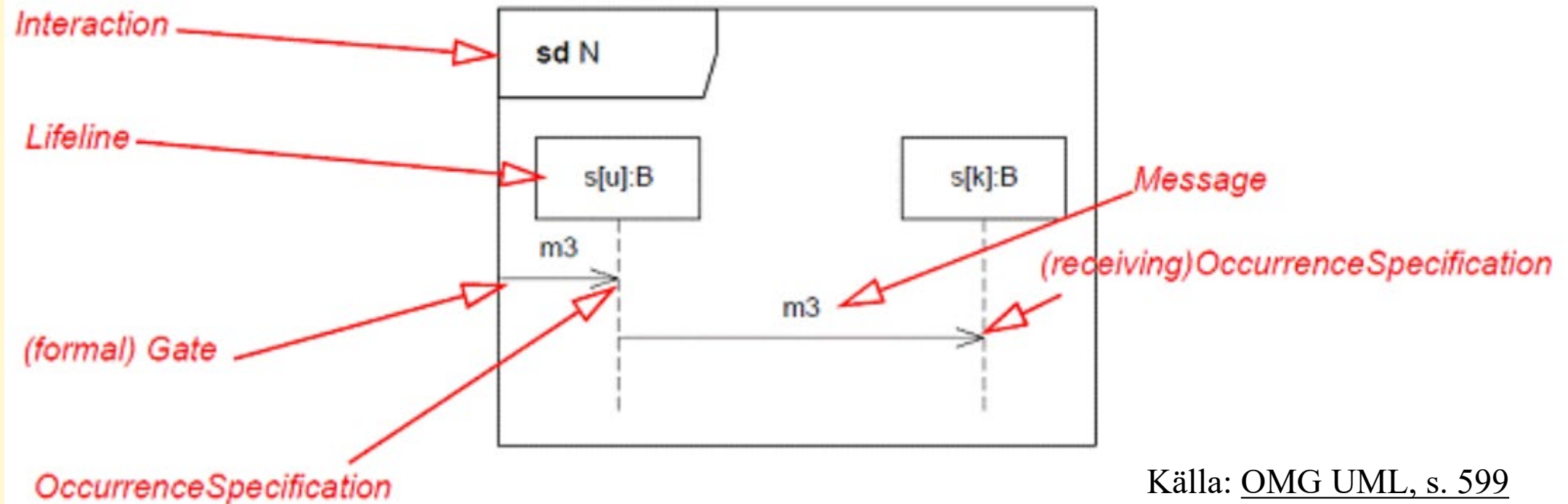
Sekvensdiagramets beståndsdelar

- Se sida 140 i kursboken
- Aktör (Actor)
- Deltagare (participant)
 - Objekt
- Pilar
- Anrop
- Returer
- Kombinerade fragment
 - Loopar
 - villkor



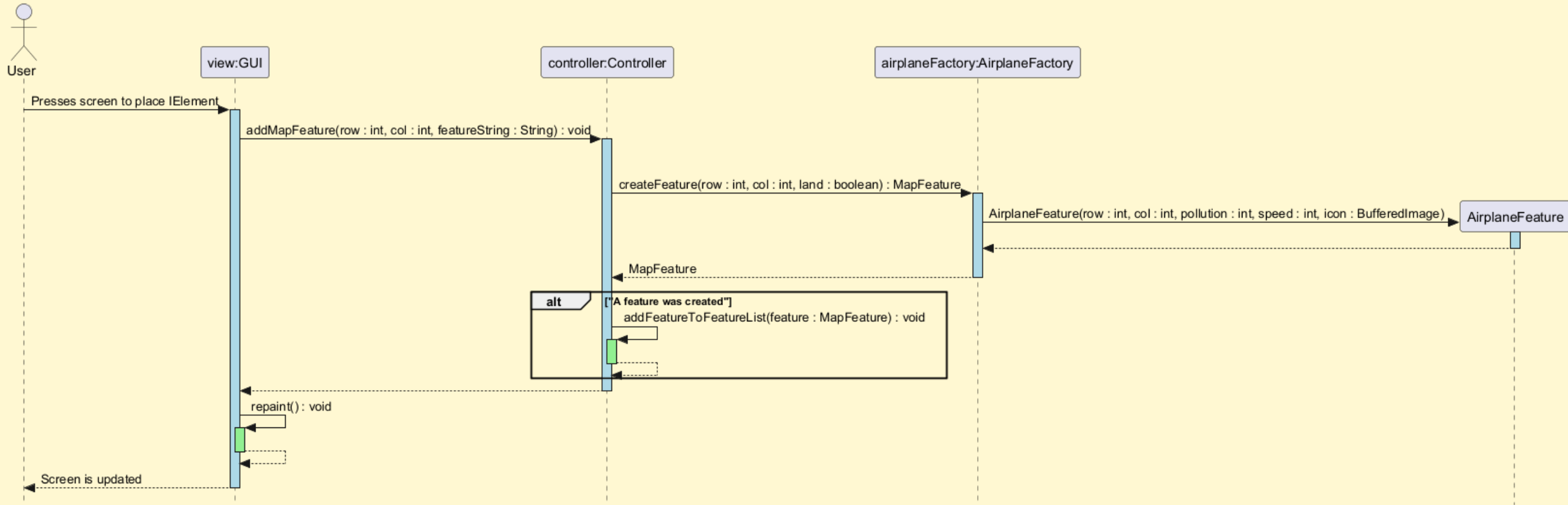
Dokumentationens *enda* s-diagram

17.8.2 Example Sequence Diagram



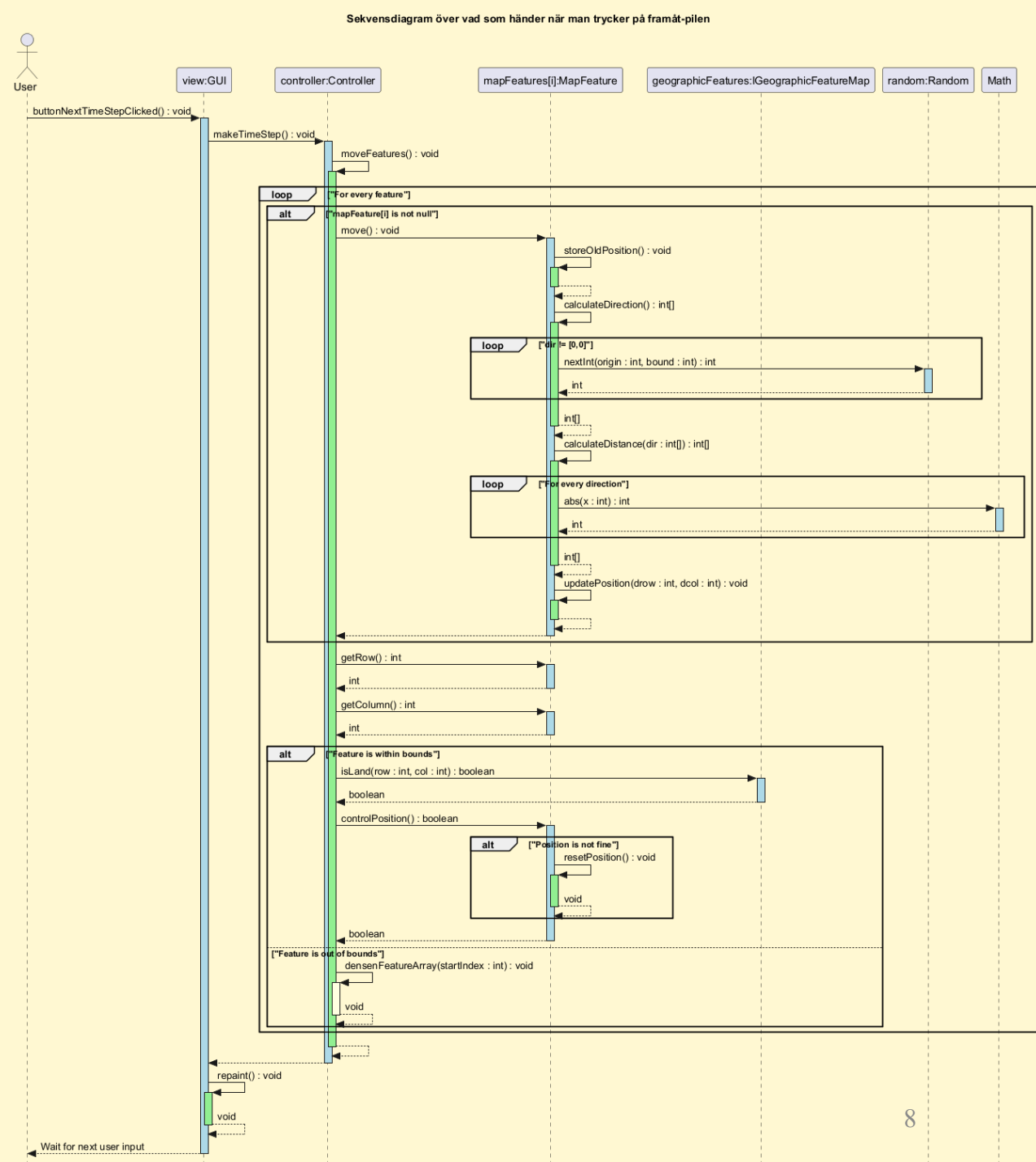
Exempel från Assignment 1

Sekvensdiagram över placering av IElement (förenklad)



Exempel från Assignment 1

- Visar vad som händer när vi trycker på framåt-pilen

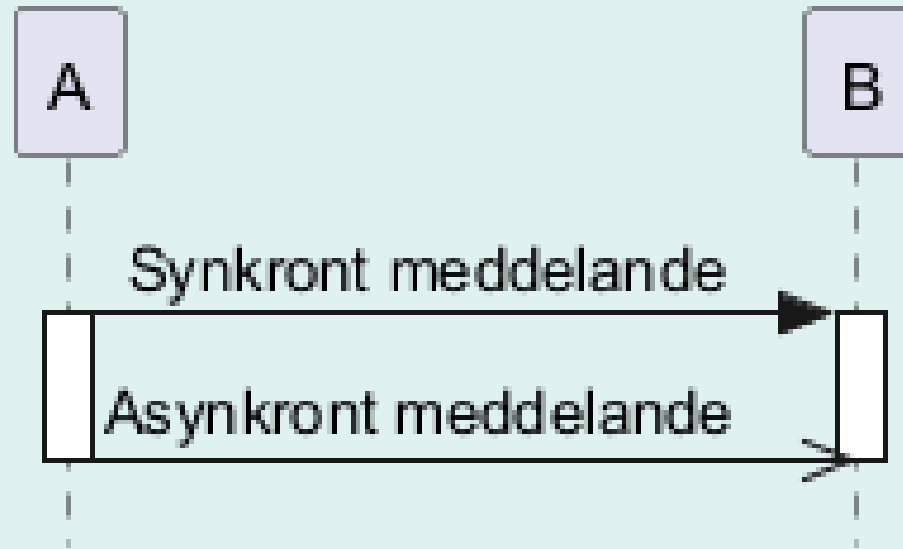




Ny notation

Synkrona- & asynkrona meddelanden

- I tidigare kurser har alla meddelanden varit *synkrona*
 - De har skett i ordning och väntat på varandra
- Med flertrådade program kan vi ha *asynkrona* meddelanden
- Programmet väntar inte på svar, utan fortsätter att exekvera
- Detta visas i sekvensdiagram med olika *form* på pilarna

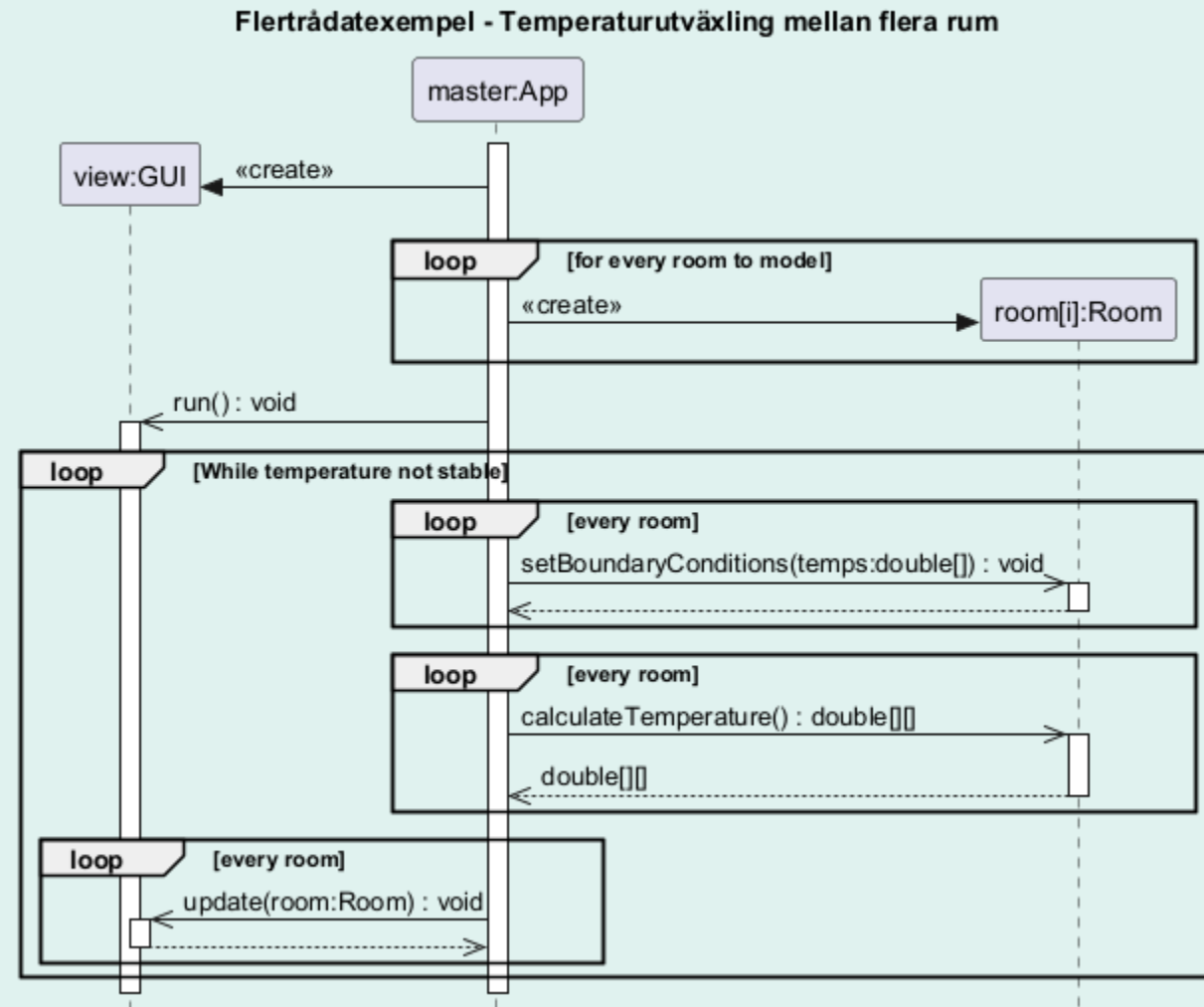


Exempel på asynkrona meddelanden

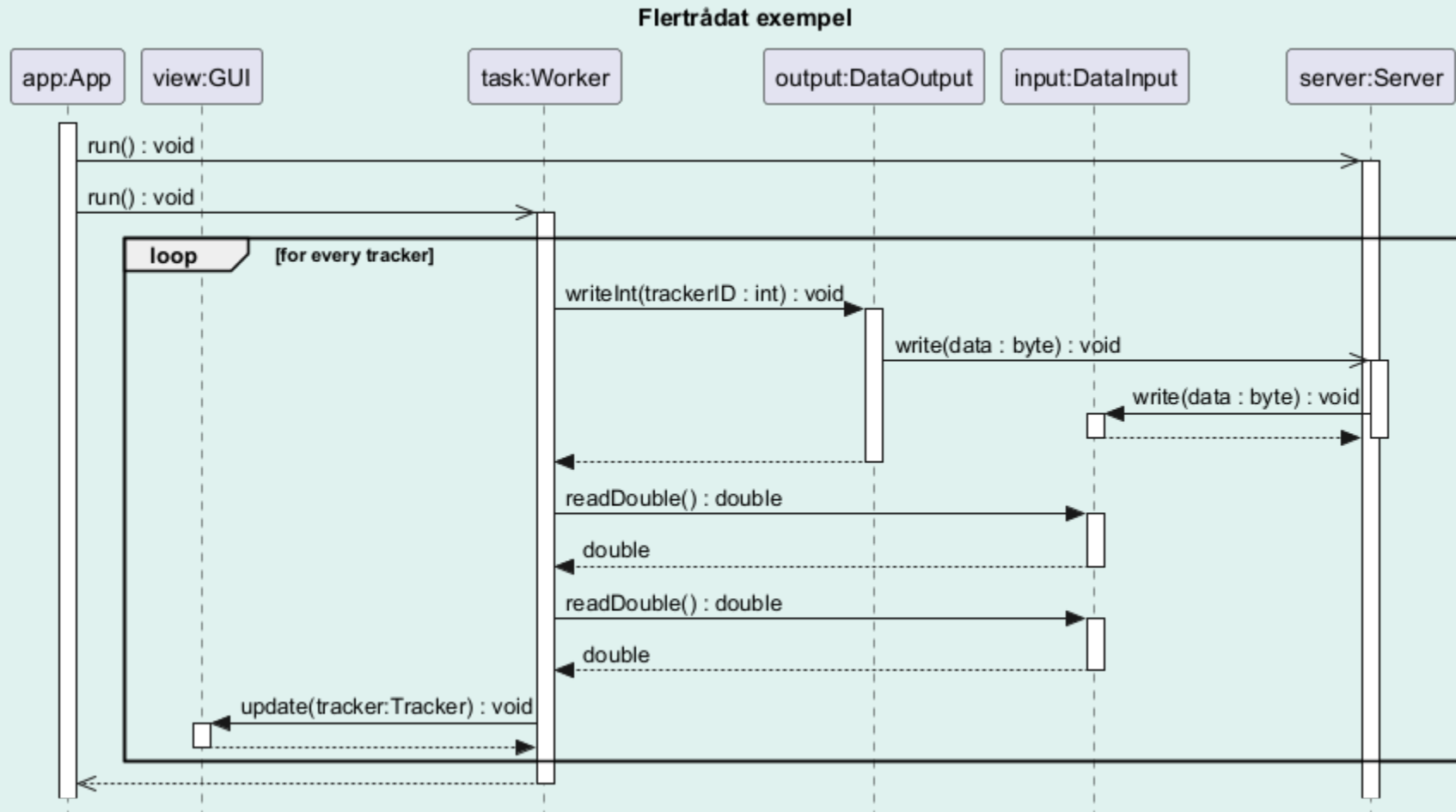
- Kommunikation via text
 - Om du skriver ett brev stannar du inte upp tills du får ett svar
 - Det samma gäller digitala meddelanden (SMS, Snapchat, o.s.v.)
 - Istället fortsätter du med annat, tills du får ett svar och hanterar det
- Matlagning
 - Om du ska koka upp ris så startar du spisen och förbereder resten av maten tills riset är klart.
- Förenklat: När du startar en ny tråd använder du ett asynkront meddelande

Min kandidatuppsats

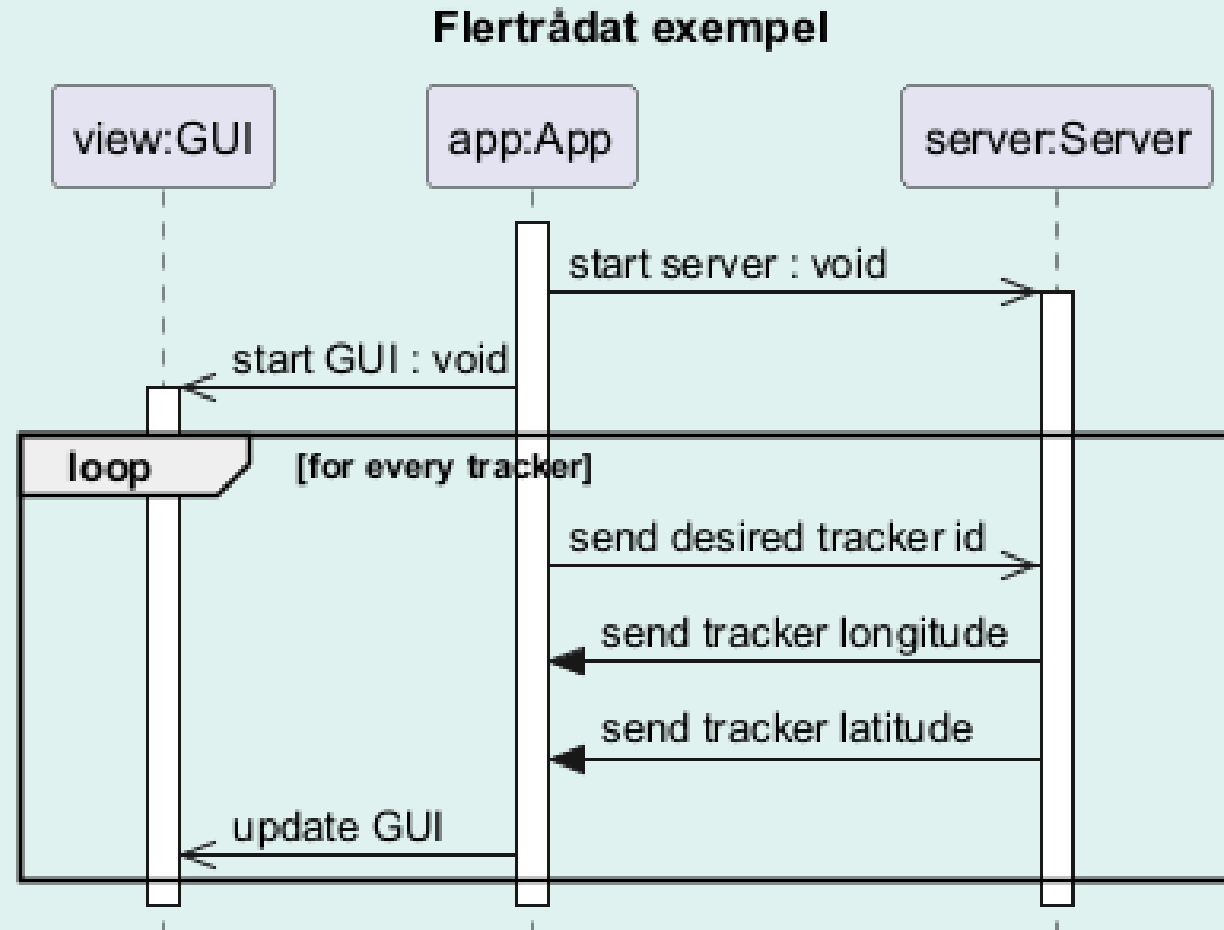
- Ett exempel från min kandidatuppsats i Numerisk analys
 - Aningens förvanskat
 - Gjordes utan klasser
- Varje rums temperatur beräknades på sin egen dator i.e. flertrådat



Ett exempel till



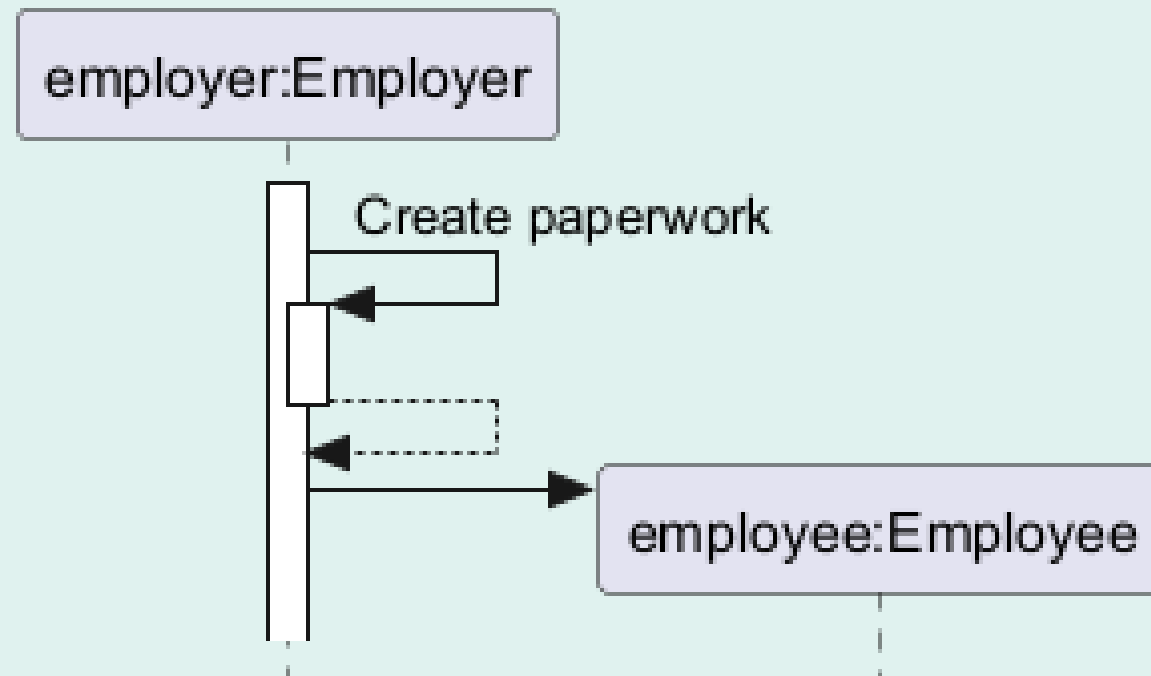
Samma exempel — utan mellanstegen



Skapa objekt

- När vi skapar ett nytt objekt använder vi en ”lämplig” pil
- Det nya objektets *lifeline* börjar där det skapas

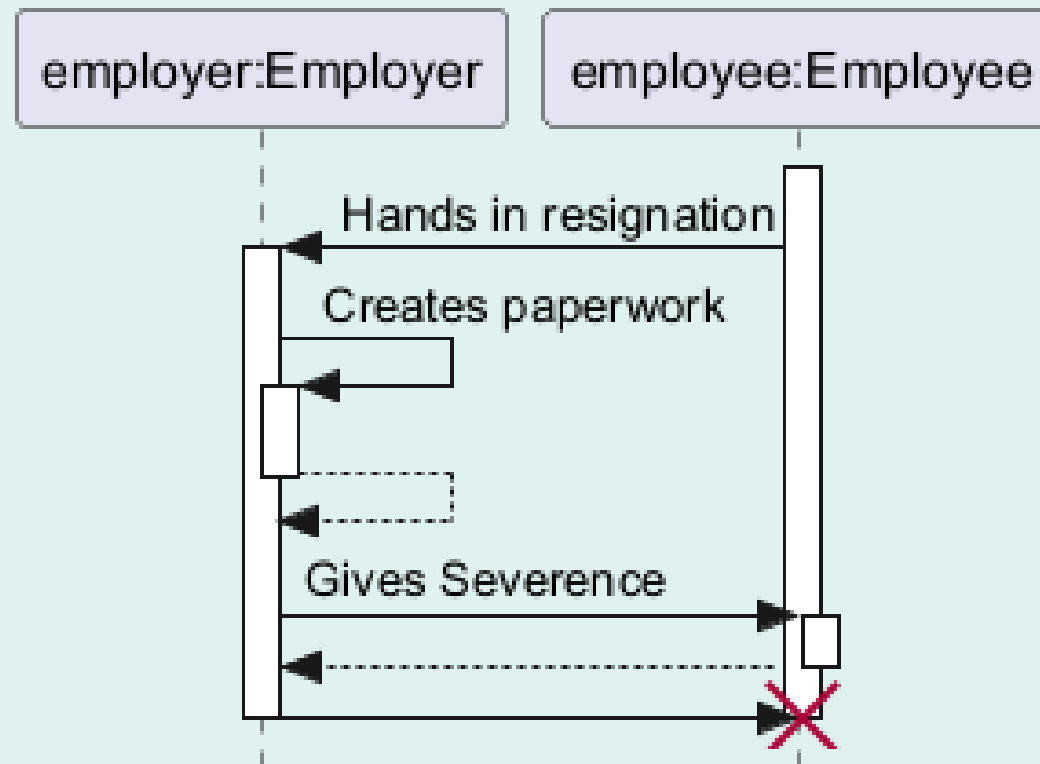
Exempel på när ett nytt objekt skapas



Förstöra objekt

- Markeras med ett kryss

Exempel på när ett objekt förstörs





Ex-tentor

Ex-tentor

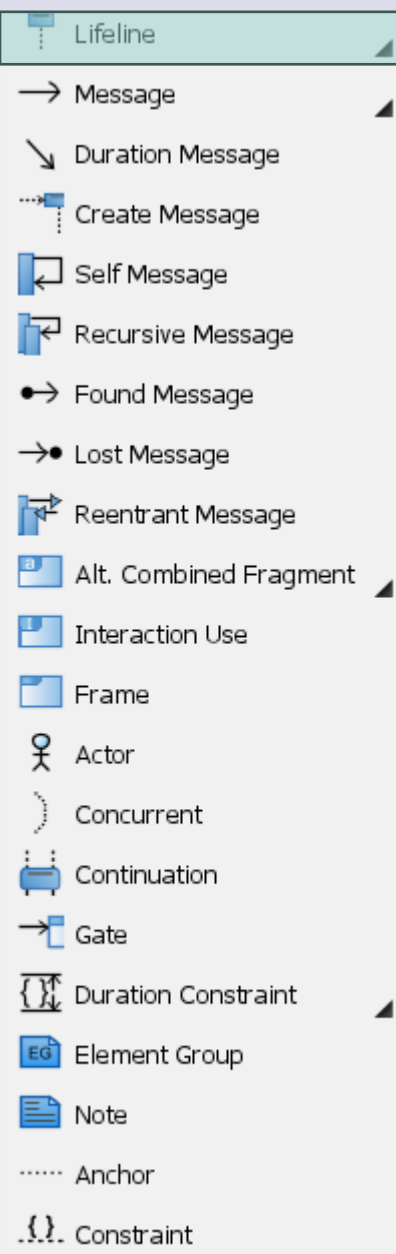
- Nu ska vi gå igenom gamla tentafrågor om sekvensdiagram
- Målet är att ta varje lösning på djupet
- Vi tar så många vi hinner
- Hellre en grundligt än flera ytligt



Visual Paradigm

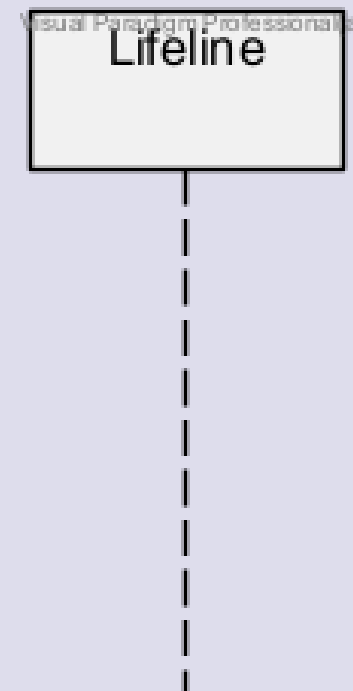
Referens för Visual Paradigm 17.2

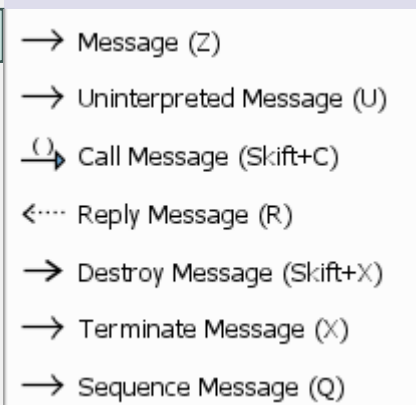
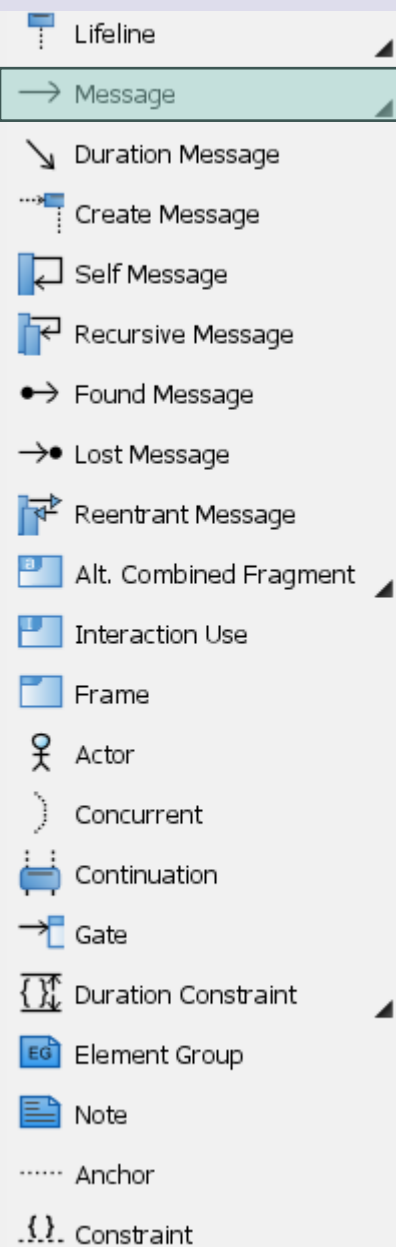
- Följande sidor är tänkta som en referens för Visual Paradigm 17.2
- Ska visa var olika knappar och funktioner sitter
- På tentamen kommer du att ha tillgång till Visual Paradigm
- Lär dig att hitta de vanligaste funktionerna
- Att skapa diagram i Visual Paradigm är mycket lättare med en datormus



Lifelines

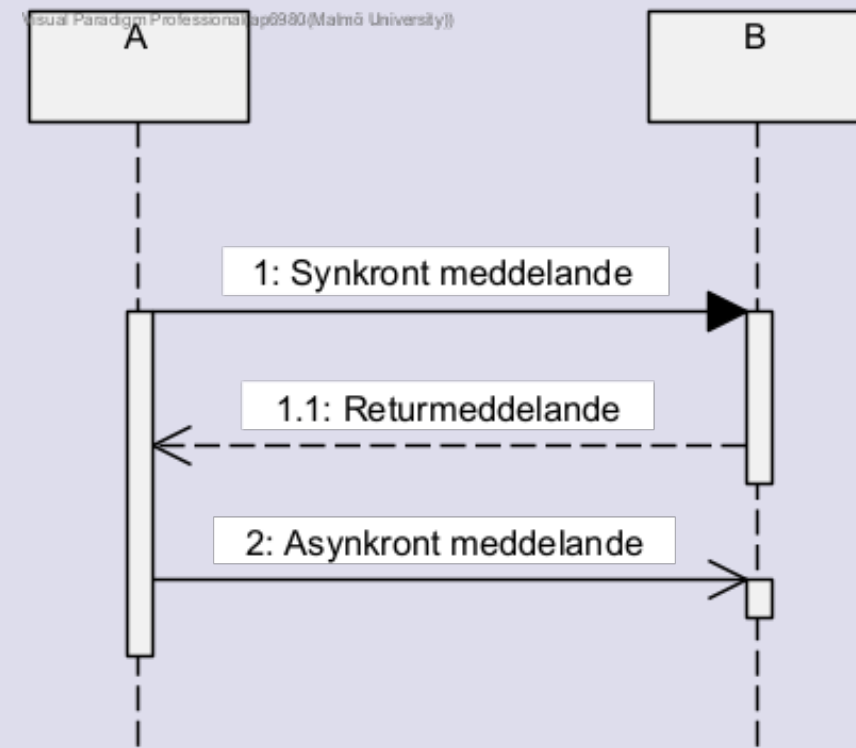
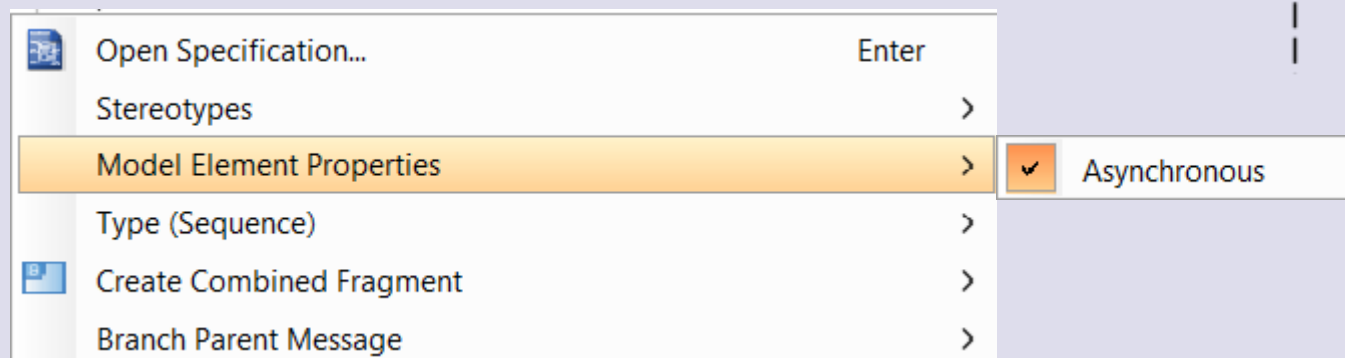
- I den här kursen gör vi inte skillnad på olika typer av objekt
 - Boundary
 - Entity
 - Controller





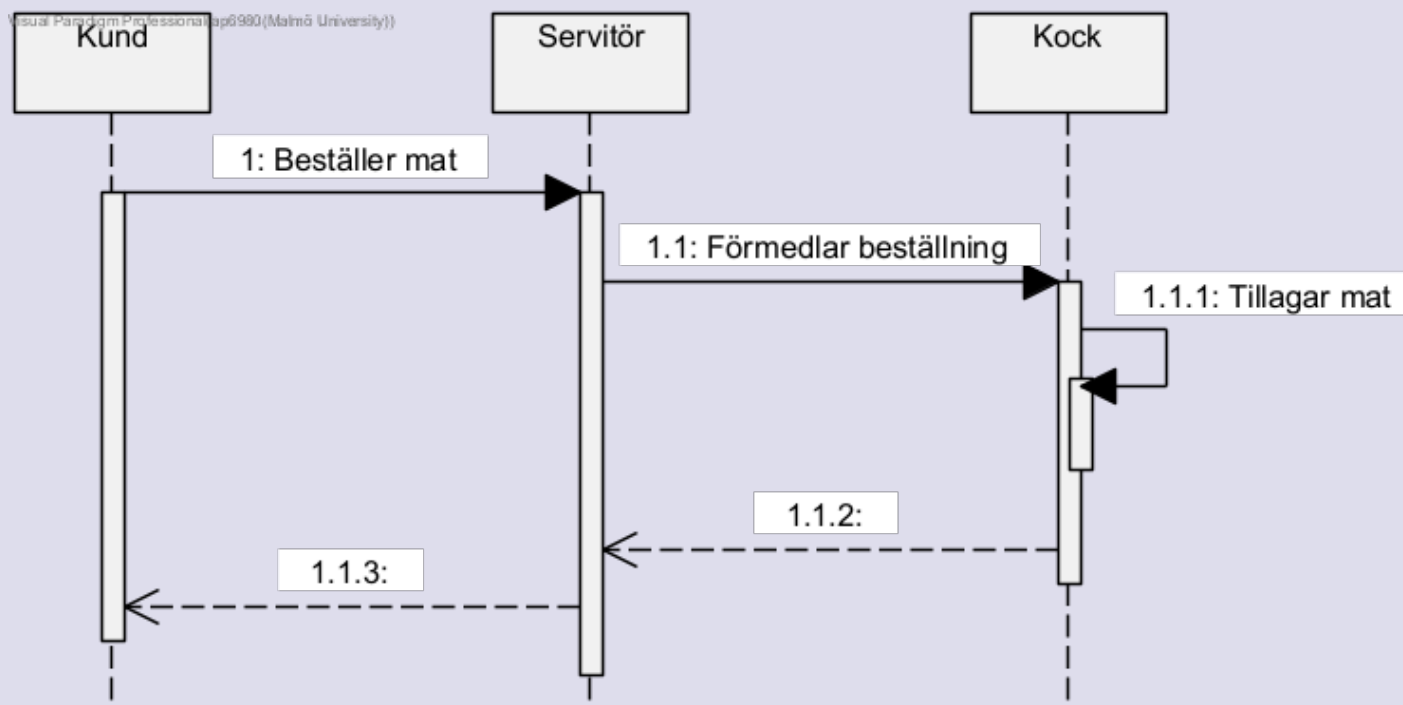
Meddelanden

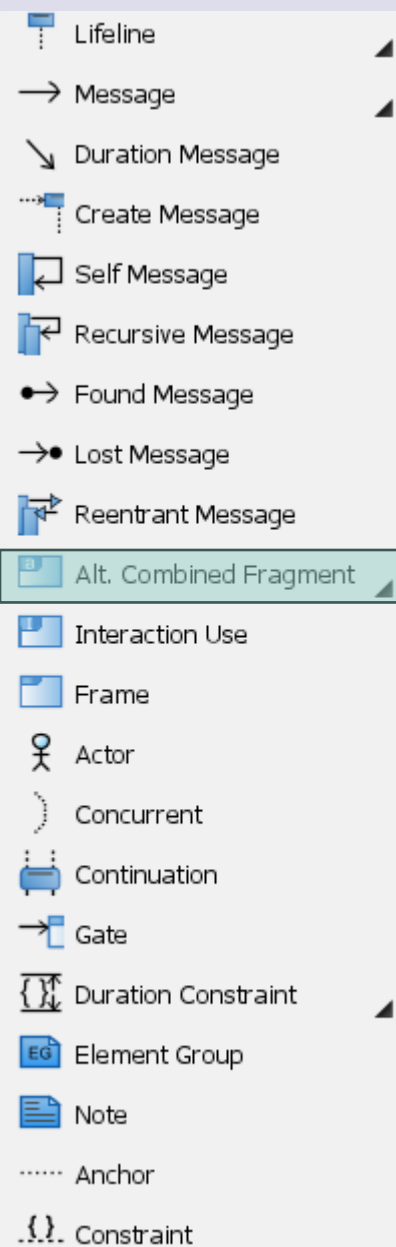
- Att skapa ett asynkront meddelande är besvärligt
- Högerklicka
- Välj *Model Element Properties*
- Klicka i Asynchronous



Självanrop

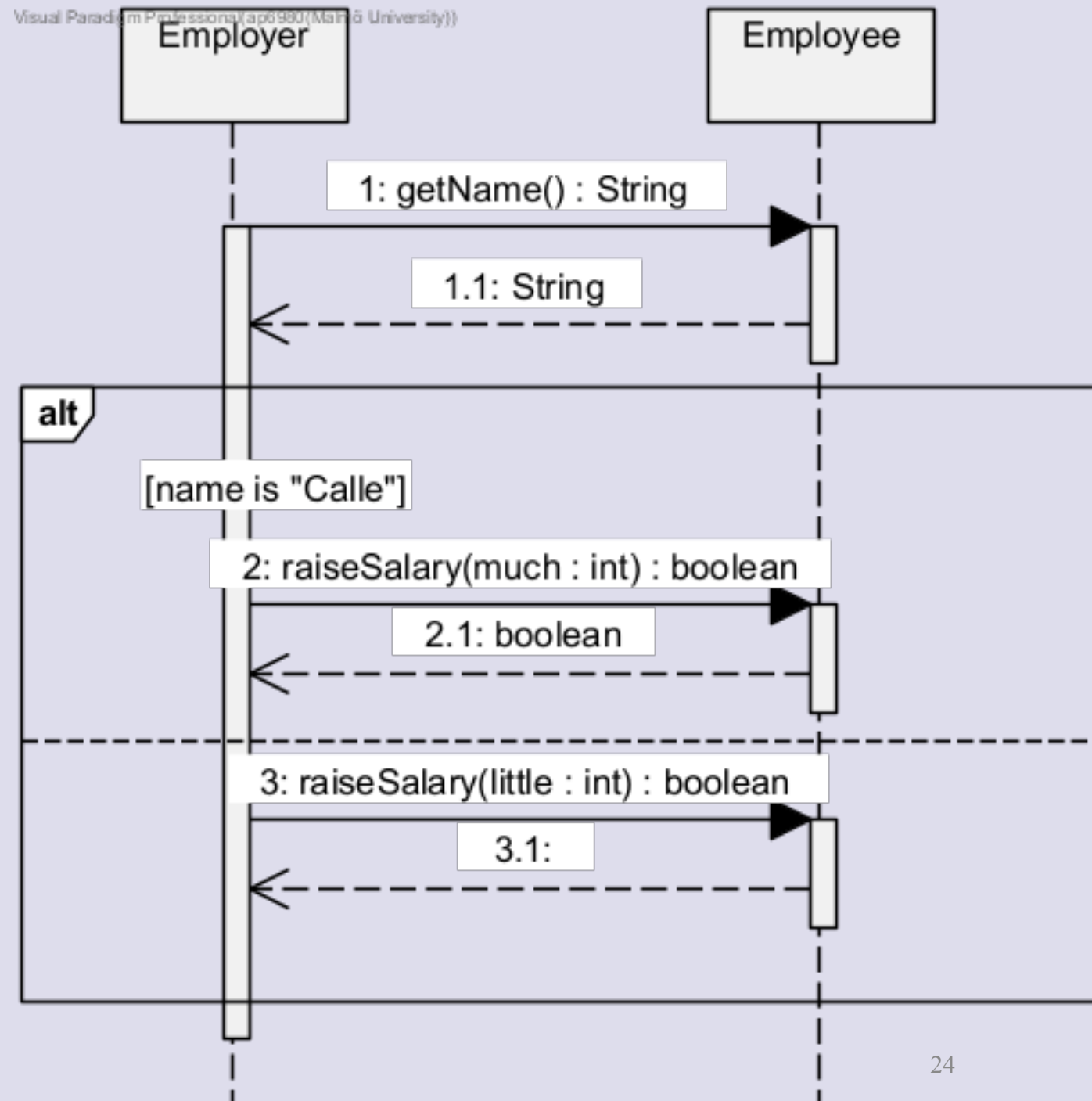
- Används när ett objekt skickar ett meddelande till sig själv
- Visual Paradigm klarar inte av att lägga till returpilar på självanrop

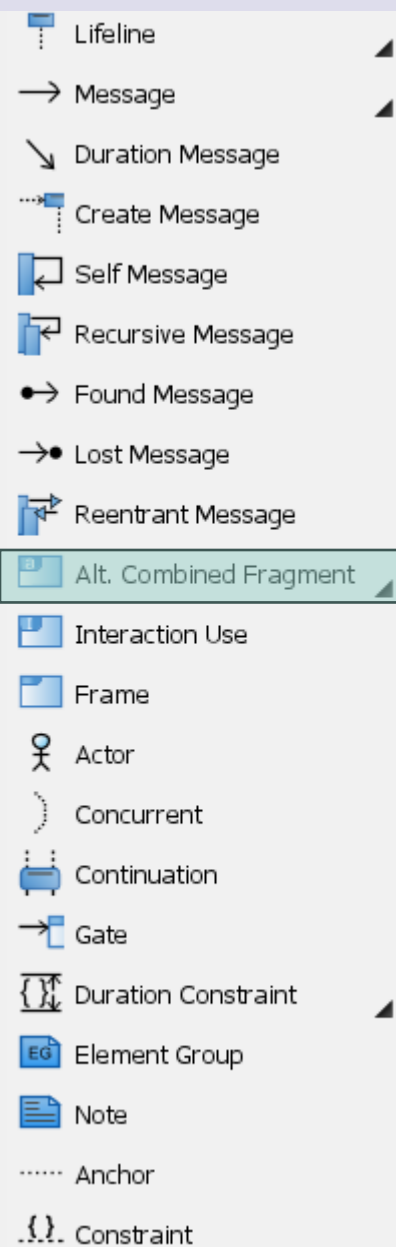




Alt

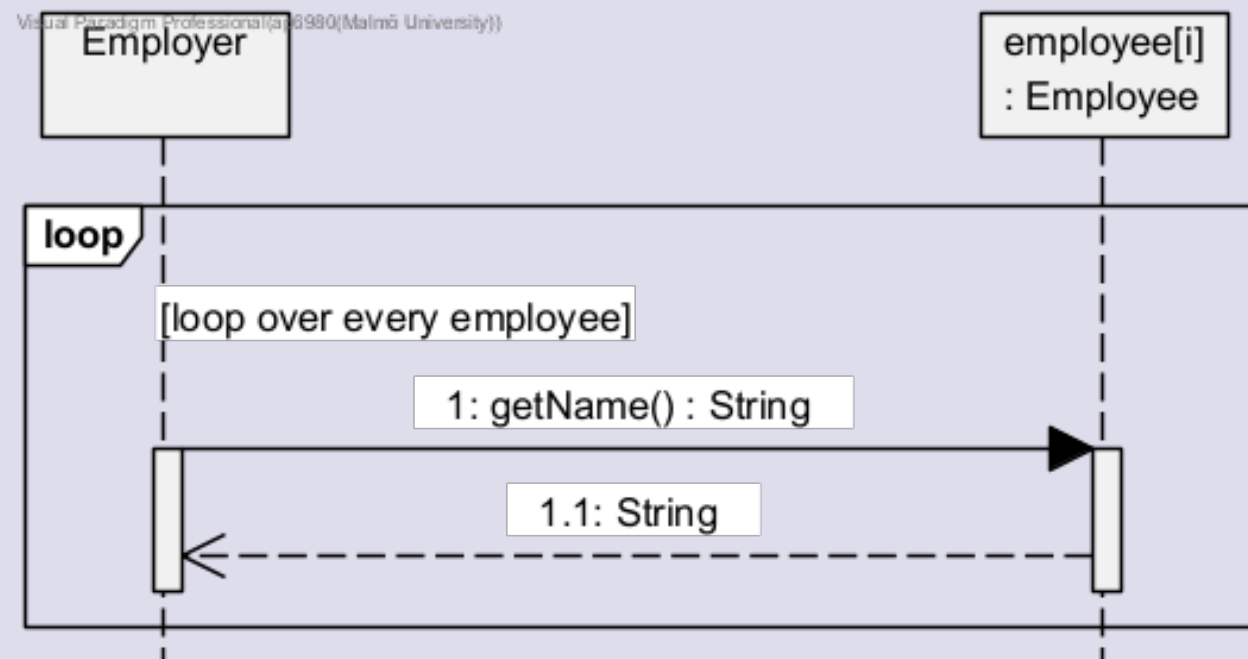
- Ett Combined Fragment
- Ska ha en *guard*
- Delar knapp med *loop*
- Alt vs. Opt
 - Alt är en *if-else*
 - Opt är en *if* utan *else*





Loop

- Är ett *Combined Fragment*
- Delar knapp med alt
- Ska ha en *guard*





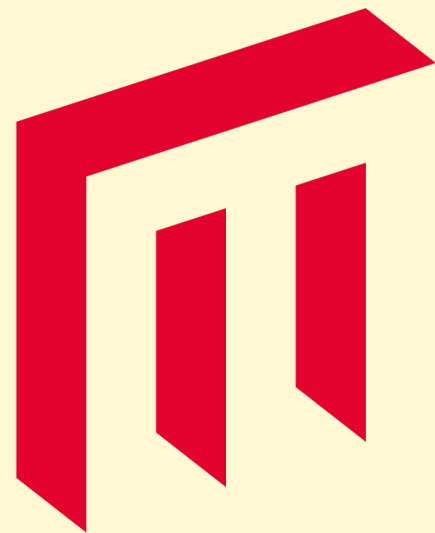
Sammanfattning

Viktiga punkter

- Sekvensdiagram visar i vilken ordning meddelanden skickas
- Vem som skickar vad till vem
- Kan visa synkrona- och asynkrona meddelanden
- Visar *inte* hur något görs

Rekommenderad läsning

- Martina Seidle, *UML @ Classroom*, kapitel 6, sidor 107–140
- Michael Ende, *Momo eller kampen om tiden*
- Object Management Group, *OMG Unified Modeling Language*, kapitel 17.8, sidor 595–599



**MALMÖ
UNIVERSITET**

